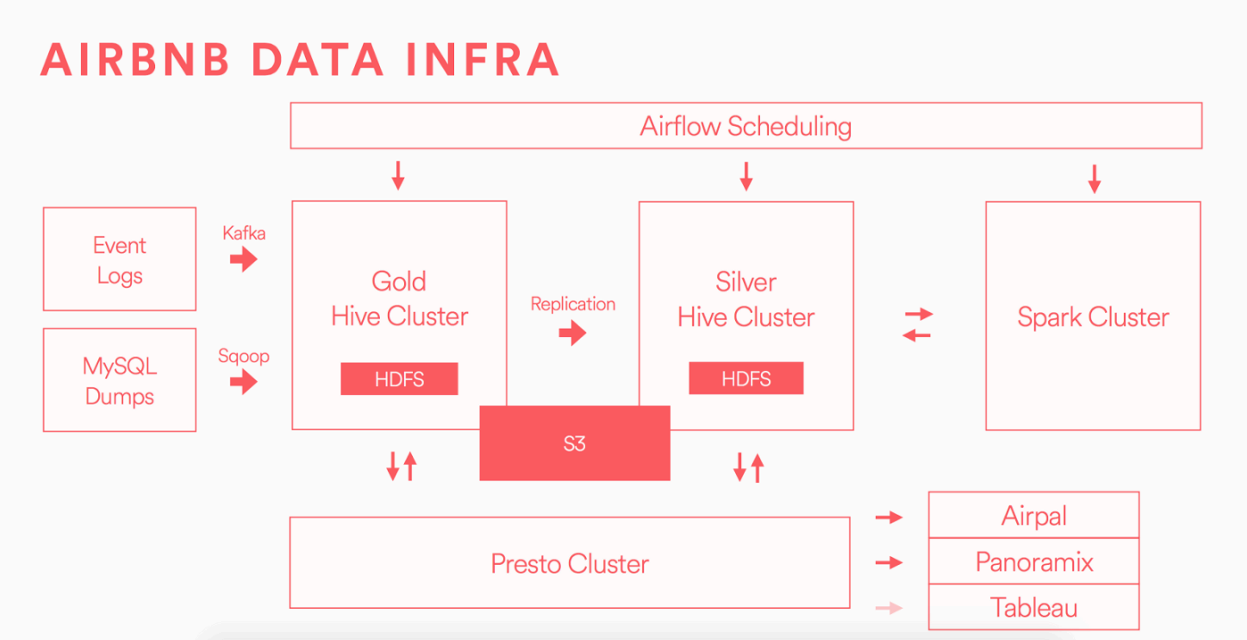
How does Apache Hive act as a data warehouse?



Airbnb created a data warehouse using Hive to achieve **reporting and analysis,** over a third of all employees at the company had launched an SQL query against Hive, just after launch.

Then What is Apache Hive?

Apache **Hive** is a **distributed**, f**ault-tolerant** data **warehouse** system that enables analytics at a massive scale and that too by writing powerful SQL queries.

This was created by **Facebook**, where they are storing

- Several thousand tables
- 700 terabytes of data
- more than 1000 users.

and is being used extensively for both reporting and ad-hoc analyses.

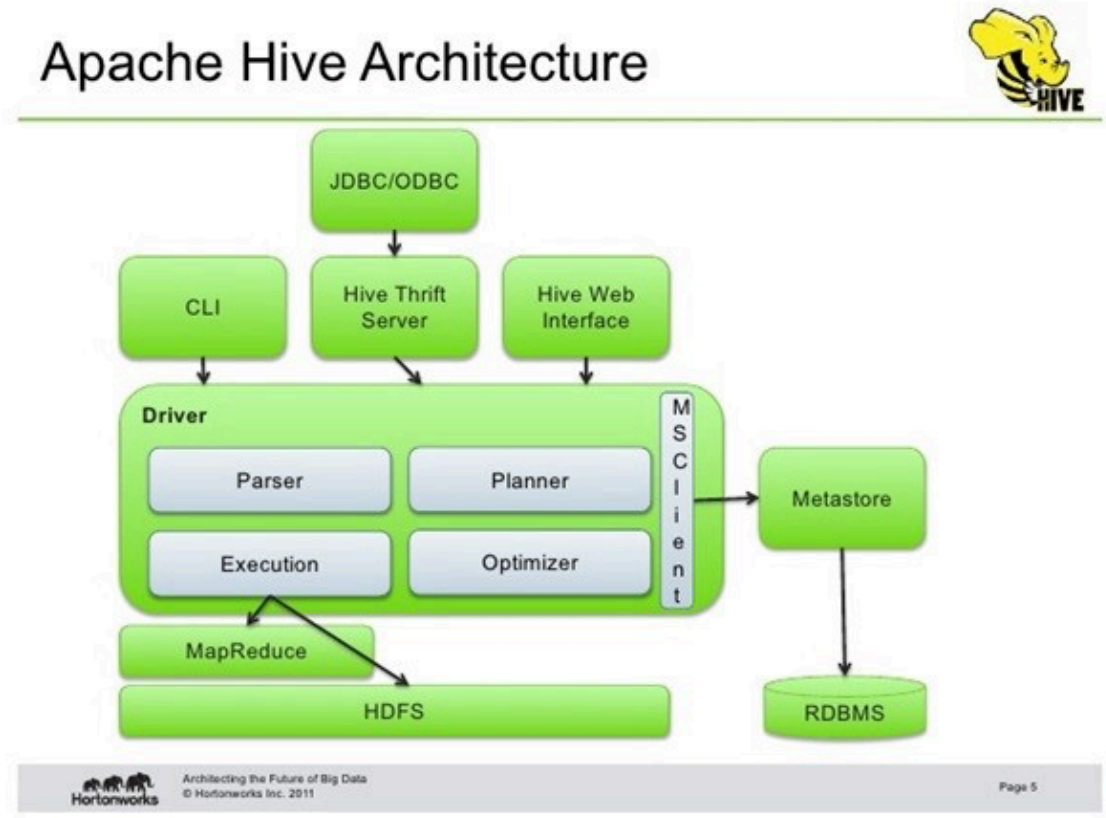
Similar to this there are giants who are **storing/processing** huge amounts of data in today's world inside Hive so how does Hive manage this?

How does Hive work under the hood? (Architecture)

All credit goes to its architecture which makes writing SQL queries possible to be executed on PB scales of data.

Main **Architecture** Components:

a. Hadoop core components(Hdfs, MapReduce)
b. Metastore
c. Hive Server
d. Driver
e. Compiler
f. Execution Engine
g. Hive Clients



a. Hadoop core components:

i. **HDFS**: When we load the data into a Hive Table it internally stores the data in HDFS path i.e by default in hive warehouse directory.

The hive default warehouse location can be found at

```
cd /etc/hive/conf
vi hive-site.xml

<property>
     <name>hive.metastore.uris</name>
     <value>thrift://nn01.jayserver.com:9083</value>
</property>

<property>
     <name>hive.metastore.warehouse.dir</name>
     <value>/apps/hive/warehouse</value>
</property>
```

ii. **MapReduce***:* By default any SQL query that we run will eventually be running a Map Reduce job by converting or compiling the query into a java class file, building a jar and executing this jar file.

```
hive (jvanchir_cards)> select count(*) from orders;

Query ID = jvanchir_20181101123144_73841f3a-3f1e-4119-8709-5c00d0bb01b9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1540458187951_3054, Tracking URL = http://rm01.         .com:19288/proxy/application_1540458187951_3054/
Kill Command = /usr/hdp/2.6.5.0-292/hadoop/bin/hadoop job  -kill job_1540458187951_3054
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2018-11-01 12:31:57,583 Stage-1 map = 0%,  reduce = 0%
2018-11-01 12:32:03,975 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 5.02 sec
2018-11-01 12:32:09,255 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 7.81 sec
MapReduce Total cumulative CPU time: 7 seconds 810 msec
Ended Job = job_1540458187951_3054
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 7.81 sec   HDFS Read: 3007957 HDFS Write: 6 SUCCESS
Total MapReduce CPU Time Spent: 7 seconds 810 msec
OK
68883
Time taken: 27.629 seconds, Fetched: 1 row(s)
```

b. **Metastore:** Namespace for tables.

i. This is a crucial part for the hive as all the **metadata** information related to the hive such as details related to the **table, columns, partitions**, location is present as part of it.

ii. Usually, the **Metastore** is available as part of the Relational databases eg: Derby,MySQL

```
vi hive-site.xml
    <property>
        <name>javax.jdo.option.ConnectionDriverName</name>
        <value>com.mysql.jdbc.Driver</value>
    </property>
    <property>
        <name>javax.jdo.option.ConnectionPassword</name>
        <value>Hj*&%$</value>
    </property>
    <property>
        <name>javax.jdo.option.ConnectionURL</name>
        <value>jdbc:mysql://jay01.abc.com/metastore</value>  // The DB name in mysql is metastore
    </property>
```

Note: In real-time the access to metastore is restricted to the Admin and specific users.

c. **Hive Server:**
   i. Enables clients to execute queries against Hive
   ii. Supports multi-client concurrency and authentication
   iii. Designed to provide better support for open API clients like JDBC and ODBC

d. **Compiler:**
   i. **Parses the query**
   ii. **Does semantic analysis** on the different query blocks
   iii. **Query expressions**
   iv. Eventually generates an **execution plan** with the help of the table and partition metadata looked up from the **metastore**.

e. **Driver** : The component in architecture that:
   i. Used for direct SQL and HiveQL access to Hive, **enabling Business Intelligence** (BI), **analytics**, and **reporting** on Hive-based data.
   ii. Efficiently **transforms** an application's SQL query into the equivalent form in **HiveQL**
   iii. **Interrogates** Hive to obtain schema information to present to a SQL-based application
   iv. Overall driver is nothing but A **bunch** of **jar** files

Mostly here, programs in java/python/c++/c/scala/.NET need to be written. But inside them it's SQL only.

What are the optimization techniques in Hive ?

Initially Airbnb started using **Apache Hive** as their core Data Warehouse as is. While Hadoop/hive can process nearly any amount Consider this as a JDBC/ODBC connection used by Java to connect to RDBMS for fetching and working on data.

f. **Execution Engine:**
   i. Execution of the execution plan made by the compiler is performed in the execution engine.
   ii. The plan is a DAG of stages.
   iii. Dependencies within the various stages of the plan is managed by execution engine
   iv. it executes these stages on the suitable system components.

g. **Hive Clients:** It is the interface through which we can submit the hive queries. Example :
   i. **Terminal interfaces** : Hive CLI(deprecated),Beeline
   ii. **Web-interfaces** : Hue, Ambari

what happened in the background from writing a query to getting a result in front of you. While the query was executing some logs like **mapreduce observed?**

SQL → MapReduce → Table(SQL)

1. First of all, the user **submits** their query and CLI sends that query to the Driver.
2. Then the **driver** takes the help of the query **compiler** to check syntax.
3. Then **compiler** requests for **Metadata** by sending a metadata request to Metastore.
4. In response to that request, metastore sends metadata to the compiler.
5. Then the compiler **resends the plan** to the driver after checking requirements.
6. The Driver sends the plan to the **execution engine**.
7. Execution engine query executes the **MapReduce** job. And in the meantime the execution engine executes metadata operations with Metastore.
8. Then the **execution engine** fetches the results from the Data Node and sends those results to the driver.
9. At last, the driver sends the results to the hive interface.

What are different HQL commands that are used commonly?

DDL Commands in Hive

CREATE	Database,Table
DROP	Database,Table
TRUNCATE	Table
ALTER	Database,Table
SHOW	Databases,Tables,Table Properties,Partitions,Functions,Index
DESCRIBE	Database, Table ,View

DML Commands in Hive

LOAD	`LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename`

INSERT	`INSERT OVERWRITE TABLE tablename1 [PARTITION (partcol1=val1, partcol2=val2 ...) [IF NOT EXISTS]] select_statement1 FROM from_statement;`
	
	`INSERT INTO TABLE tablename1 [PARTITION (partcol1=val1, partcol2=val2 ...)] select_statement1 FROM from_statement;`

UPDATE	`UPDATE tablename SET column = value [, column = value ...] [WHERE expression]`
DELETE	`DELETE FROM tablename [WHERE expression]`
EXPORT	`EXPORT TABLE tablename [PARTITION (part_column="value"[, ...])]` `  TO 'export_target_path' [ FOR replication('eventid') ]`

IMPORT	`IMPORT [[EXTERNAL] TABLE new_or_original_tablename [PARTITION (part_column="value"[, ...])]]`
	`  FROM 'source_path'`
	`  [LOCATION 'import_target_path']`

of data, **optimizations** can lead to big savings, proportional to the amount of data, in terms of processing time and cost.

Below are the optimizations that are recommended :

a. Execution Engine in Hive

Later on with time , the Airbnb team realized that existing Hive architecture needs to be optimized in order to fully harness it and they started working on optimizing the query engine first as MR was getting stale. Below are the options:

- **Apache Tez** :**Apache Tez** is a client-side library which operates like an execution engine, an alternative to traditional **MapReduce Engine**.

  set hive.execution.engine=tez;

- **LLAP(Long Lived Analytical Processing):**a SQL-on-Hadoop processing framework, bringing the promise of low latency SQL queries on Hadoop and YARN.
- **Hive on Spark:** Latest and recommended, provides Hive with the ability to utilize Apache Spark as its execution engine. Will cover Spark in detail

—> IN spark add hive too

b. Partitioning

Problem Statement:
Airbnb marketing decided to review their targets every biweekly on every Monday(gap of 14 days) where they will strategize and bring efficient decisions to the table.

To finalize this, they need data on every 14th and 28th of every month from hive.But every time scanning billions of rows doesn't make sense. Unnecessary scanning of tables can lead to latency with eventually query failing.

Proposed Solutions :

Option 1 : Create a separate table for each date every month before the marketing team asks?
Option 2: Let's divide the table into parts based on the values of particular columns**.**

Deserving Solution: Best way to go ahead is with Option 2

So what **Option 2** talks about is **Partitioning**.

- Partitioning divides the table into parts based on the values of particular columns.
- A table can have multiple partition columns to identify a particular partition.
- Using partitions it is easy to do queries on slices of the data.
- The data of the partition columns are not saved in the files
- When we query data on a partitioned table, it will only scan the relevant partitions to be queried and skip irrelevant partitions.
- Partitioning is basically of two types: **Static** and **Dynamic**.

Static partitioning:

- Manually decide how many **partitions** tables will have and also value for those partitions
- Here tables are populated by using the **LOAD** command to create the partition.
- Usually when loading files (big files) into Hive tables static partitions are preferred.
- Enable Static partition in the hive as:
    - **set hive.mapred.mode = strict**
- Always use **where** clause to use limit in the static partition.

Dynamic partitioning:

- This provides flexibility and creates partitions automatically depending on the data that we are inserting into the table.
- INSERT with a SELECT query are used to create multiple partitions in table sales dynamically.
- If there is a requirement to partition a number of columns but you don't know **how many columns** then also **dynamic partition** is suitable.
- Here, there is no mandatory need of **where clause**.
- To enable dynamic partitioning:
    - **set hive.exec.dynamic.partition=true;**
    - **set hive.exec.dynamic.partition.mode=nonstrict;**

When to use Partitioning?

1. When reading the entire data set takes too long
2. When queries almost always filter on the partition columns
3. When there are a reasonable number of different values for partition columns

When Not to Use Partitioning

1. When columns have too many unique rows
2. When creating a dynamic partition as it can lead to high number of partitions
3. When the partition is less than 20k

Bucketing

Problem Statement:
Airbnb marketing decided to review their targets every biweekly on every Monday(gap of 14 days) and this time they added that data should be given company verticals, for example :

a. Get data from sales team
b. Get data from Infrastructure team
c. Get data from customer care team

To finalize this, they need data on every 14th and 28th of every month from hive.But every time scanning billions of rows doesn't make sense. Unnecessary scanning of tables can lead to latency with eventually query failing.

Proposed Solution:

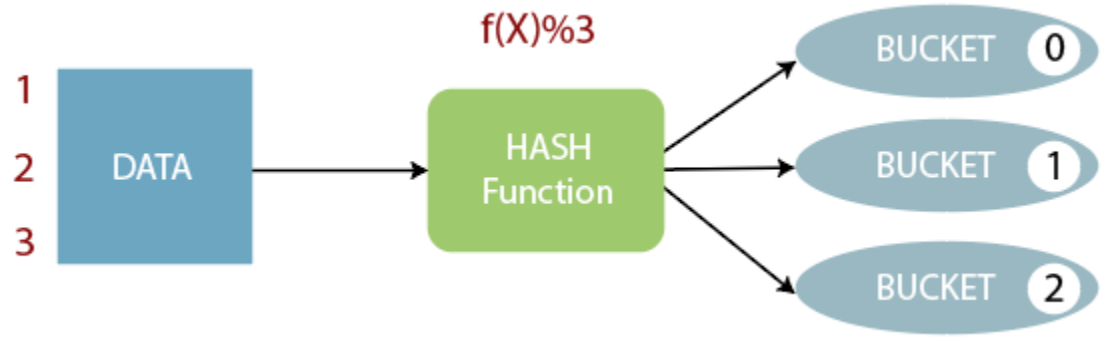
a. Partitioning can be used to group data this way
b. Groupby statement can be used while issuing query
c. Segregating hive table data into multiple files/directories.

Solution:

Option a : as we already discussed partitioning above and it clearly says that data can be distributed as partition across clusters but inside partition there is no way to group.
Option b : Group By statement is used at client side to get data segregated. Here we dont want TB/PB of data to be streamed online and then gets groped

Option c: It's better to segregate data into multiple files and this is what is called **bucketing.**

Bucketing: provides flexibility to further segregate the data into more manageable sections called buckets or clusters and this is how it works:



a. The concept of bucketing is based on the hashing technique.
b. Here, modules of current column value and the number of required buckets is calculated (let say, F(x) % 3).
c. Now, based on the resulting value, the data is stored into the corresponding bucket.
d. **Records which are bucketed by the same column value will always be saved in the same bucket.**

Using AVRO/ORC/Parquet Format

By default, **text file**(csv,tsv) is the default format supported by hive. Yes!! This is human friendly (easy to read) and it has support for read/write in any programming language but is not recommended for big data as:

a. Consumes more space to store numeric value as string
b. Difficult to represent in binary form

Avro was born to be used in Hive and this has major improvements like:

- Embeds schema metadata in file
- Supports **JSON** format
- Serializing data in a compact binary format
- Ideal for long term storage.
- Efficient storage

SAMPLE AVRO FILE:

```
{
  "name": "cia.sad.Operative",
  "type": "record",
  "fields": [
    {
      "name": "name",
      "type": "string"
    },
    {
      "name": "birthdate",
      "type": {
        "type": "int",
        "logicalType": "date"
      }
    },
    {
      "name": "operation",
      "type": {
        "name": "cia.sad.Operation",
        "type": "enum",
        "symbols": ["SILVERLAKE", "TREADSTONE", "BLACKBRIAR"]
      }
    }
  ]
}
```

Disadvantages :

1. AVRO supports row wise storage. For ex: in below table if we need information about ID = 3, first 2 rows will be scanned unnecessarily hene more I/O

ID	Name	Department
1	emp1	d1
2	emp2	d2
3	emp3	d3

The data in a row-wise storage format will be stored as follows:

1	emp1	d1	2	emp2	d2	3	emp3	d3

2. AVRO is not that much fit for Analytical queries.
3. To read/write data, everytime schema is needed.

Parquet File Format:

And then came a columnar file format which is :

1. More efficient
2. Improves query performance
3. Store data with nested structures
4. Low storage consumption.

Example:

1	2	3	emp1	emp2	emp3	d1	d2	d3

Cost-based Optimizer

There are two major optimizers now in Hive to further optimize query performance in general,

a. Query Vectorization :
    i. Allows Hive to process a batch of rows together instead of processing one row at a time
    ii. Operations are performed on the entire column vector, which improves the instruction pipelines and cache use

iii. To enable vectorization,

SET hive.vectorized.execution.enabled=true; -- default false

b. Cost-Based Optimization (CBO):
   i. A cost-based optimizer (CBO) generates efficient query plans.
   ii. CBO provides two types of optimizations: logical and physical.
   iii. The CBO uses column and table statistics stored in the Hive metastore to create query plans that improve query performance.
   iv. Statistics for the CBO are objects that contain information about the columns and partitions of a table.
   v. The CBO uses these statistics to estimate the cardinality and number of distinct values, etc. These estimates then help generate highly efficient query plans.
   vi. To enable it :
      1. **SET hive.cbo.enable=true** -- default true after v0.14.0